

Part 1

Public Edge Impulse project link: <https://studio.edgeimpulse.com/public/994179/live>

1. Raw inputs: State the raw inputs used in the Edge Impulse project.
The raw inputs used in the Edge Impulse project are accX, accY, and accZ. These values correspond to acceleration from the chest sensor in the X, Y, and Z axes.
2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.
The type of features passed into the NN classifier are spectral features. The number of input features is 33. Five feature names used to train the classifier are (1) accX RMS, (2) accX Peak 1 Freq, (3) accX Peak 1 Height, (4) accX Peak 2 Freq, and (5) accX Peak 2 Height.
3. NN classifier outputs: State the outputs of the NN classifier.
The outputs of the NN classifier are “nominal” and “off”. These represent the two states of the fan, with “nominal” representing the fan being on and “off” representing the fan being off.
4. Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.
There are three features being passed into the anomaly detector that are used to train it. All three features are spectral features. The features are (1) accX Peak 1 Height, (2) accY Spectral Power 2.0 - 5.0, and (3) accZ Spectral Power 2.0 - 5.0.
5. Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 563 us, anomaly 414 us, postprocessing 50 us
#Classification predictions:
  nominal: 0.234375
  off: 0.765625
Anomaly prediction: -0.410986
```

6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.
 - DSP 49ms, inference 563us, and anomaly 414us are all timing results. These values illustrate that the signal processing took 49ms, the NN classifier ran in 563 microseconds, and the anomaly detection took 414 microseconds.
 - Classification predictions:
 - nominal: 0.234 (23.4% chance the fan is running normally)

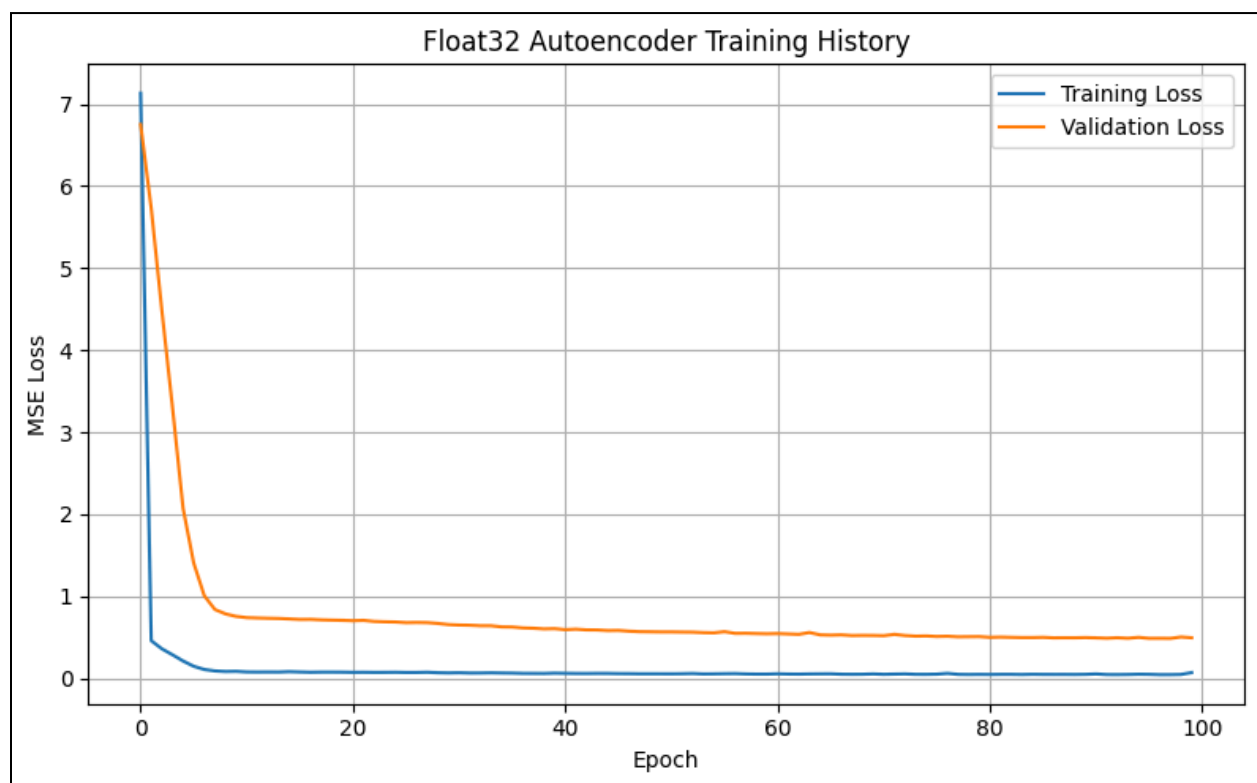
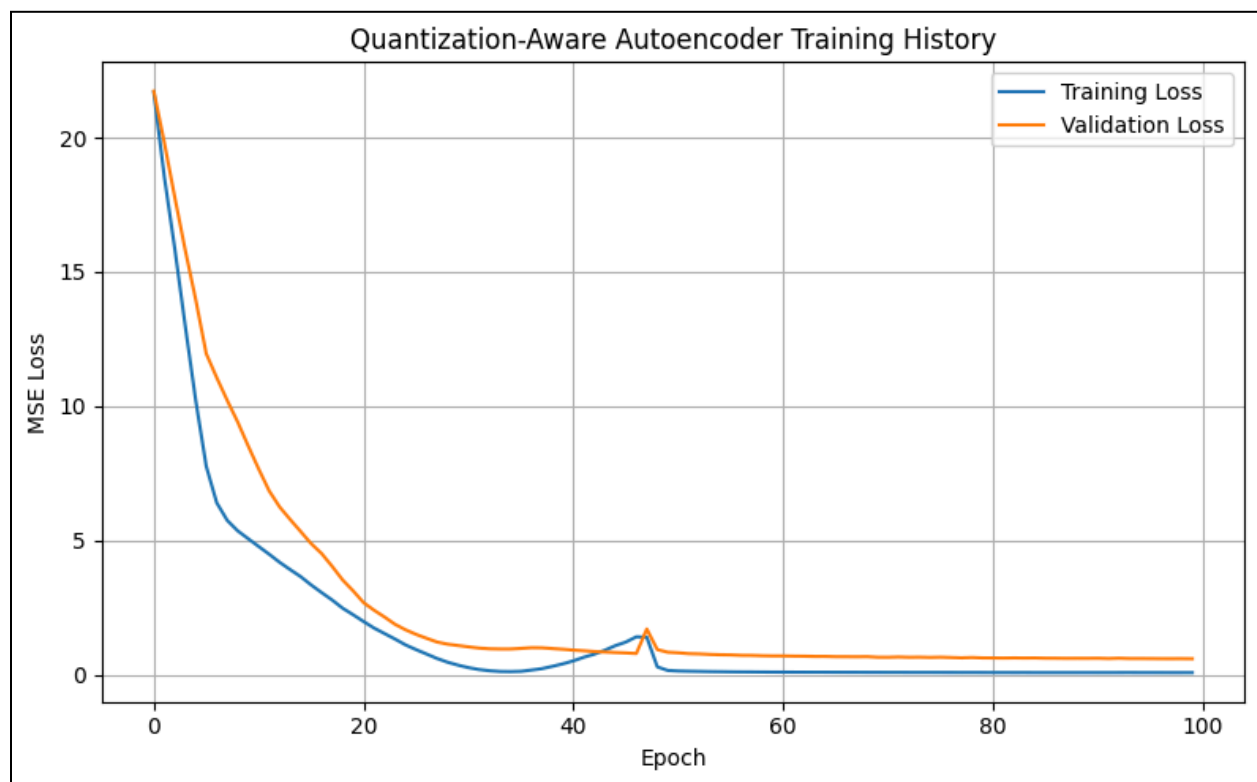
- off: 0.765 (76.6% chance the fan is off)
 - Based on this, we can conclude that the classifier thinks the fan is off
- Anomaly prediction: -0.410986
 - The anomaly score being negative indicates that the input is close to a known cluster, so it's not an anomaly. A high positive score would indicate an anomaly.

The classifier output illustrates that the fan is most likely off (with a percentage of 76.6%). The negative anomaly value indicates that the reading is within expected normal operating boundaries. The classifier is reliable when inputs closely resemble training data (normal/off fan states in similar conditions). However, when results stray from training data, our classifier may not be able to tell. For example, if there is any environmental noise that does not mimic training data, the classifier might incorrectly mark the reading as nominal or off.

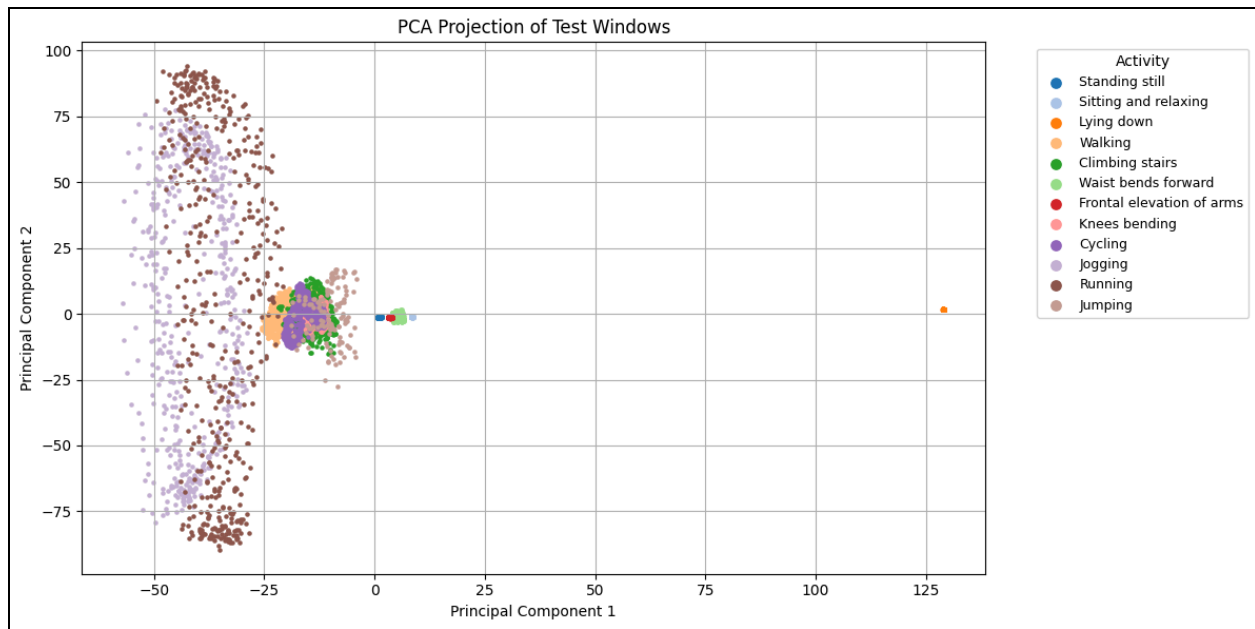
Part 2

Clear and labeled screenshots:

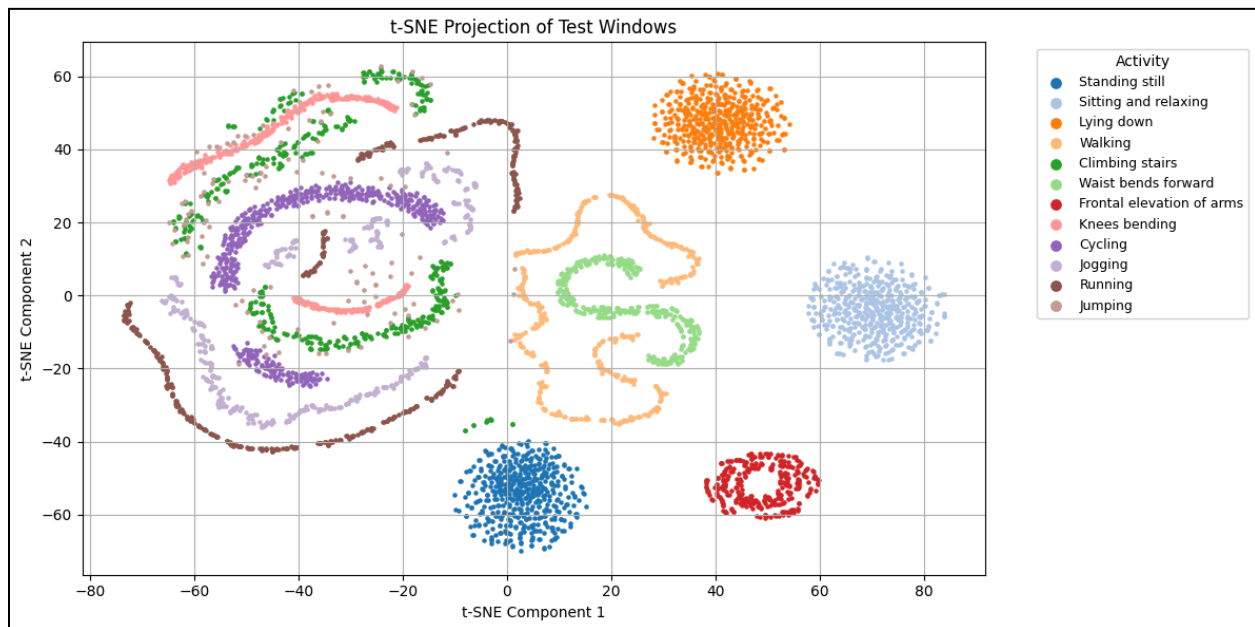
- Training and validation loss curves



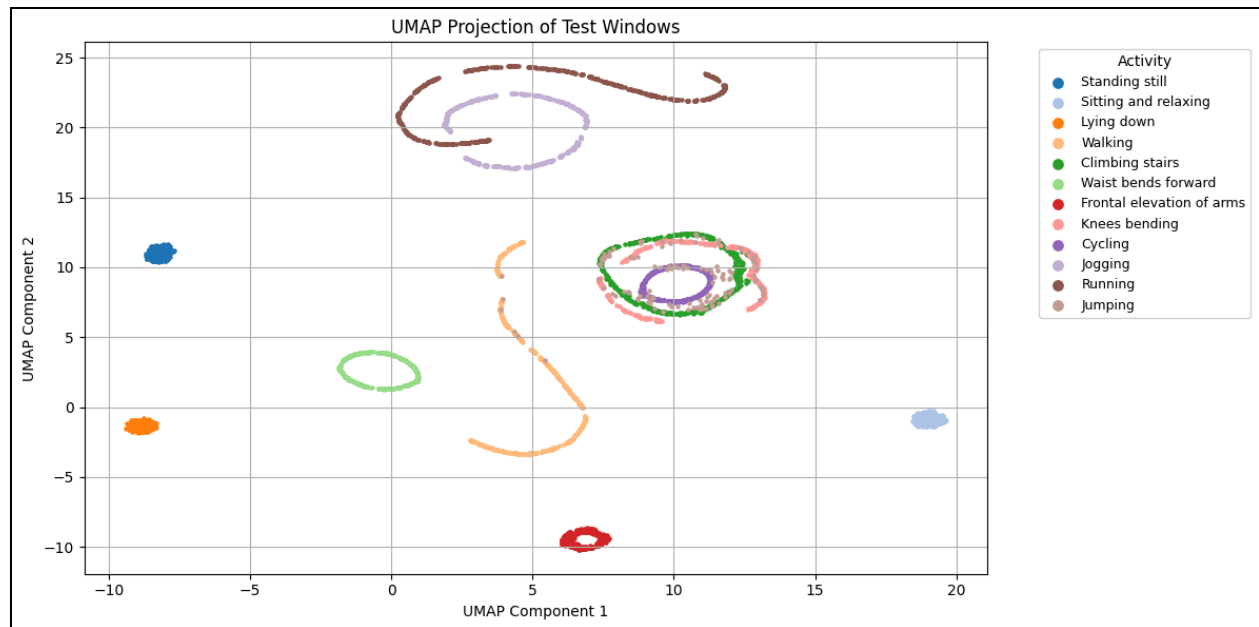
- PCA visualization



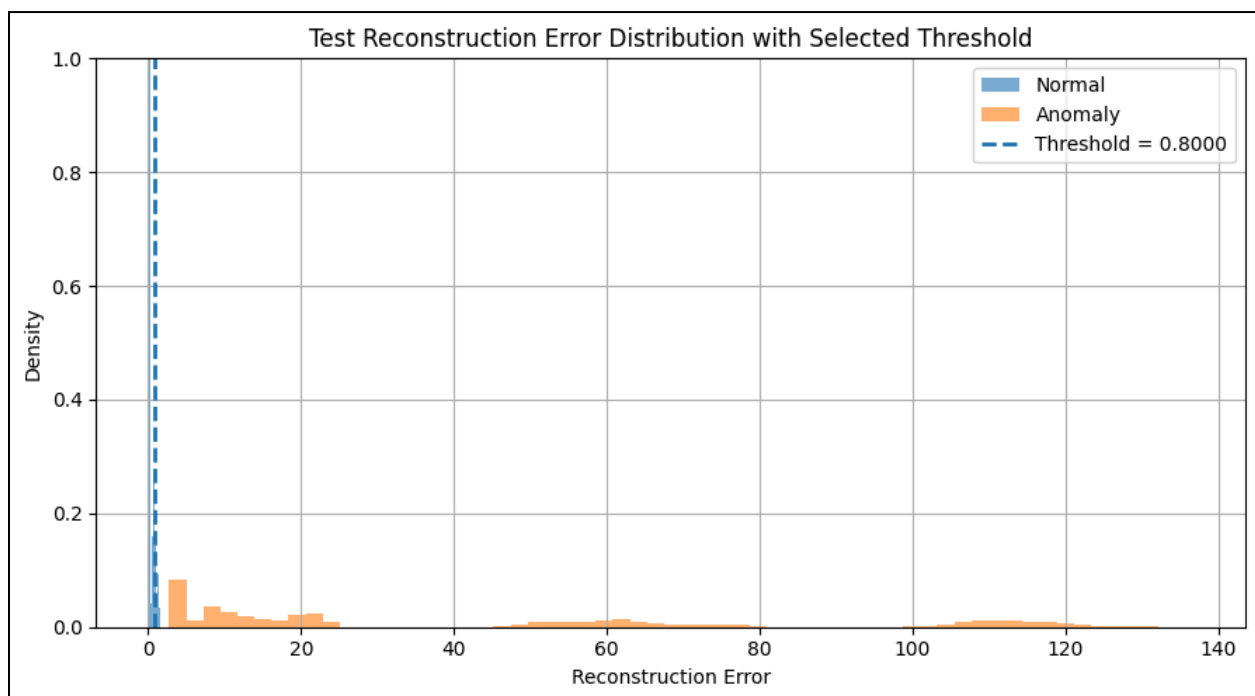
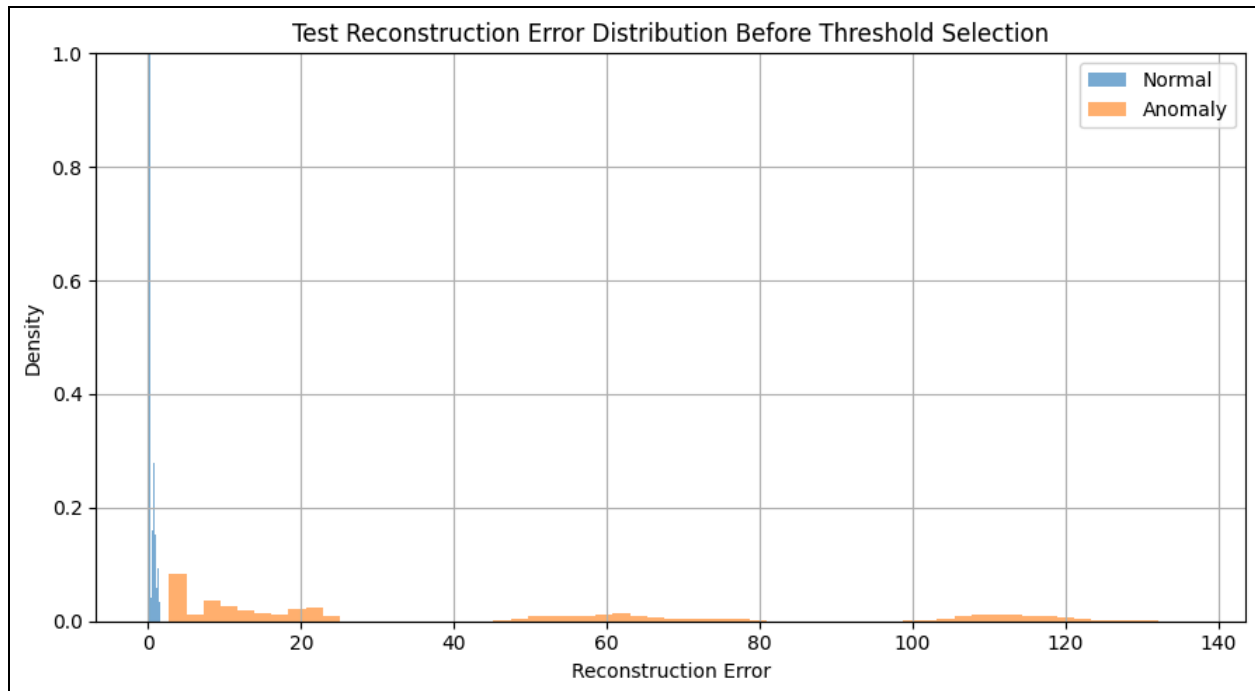
- t-SNE visualization



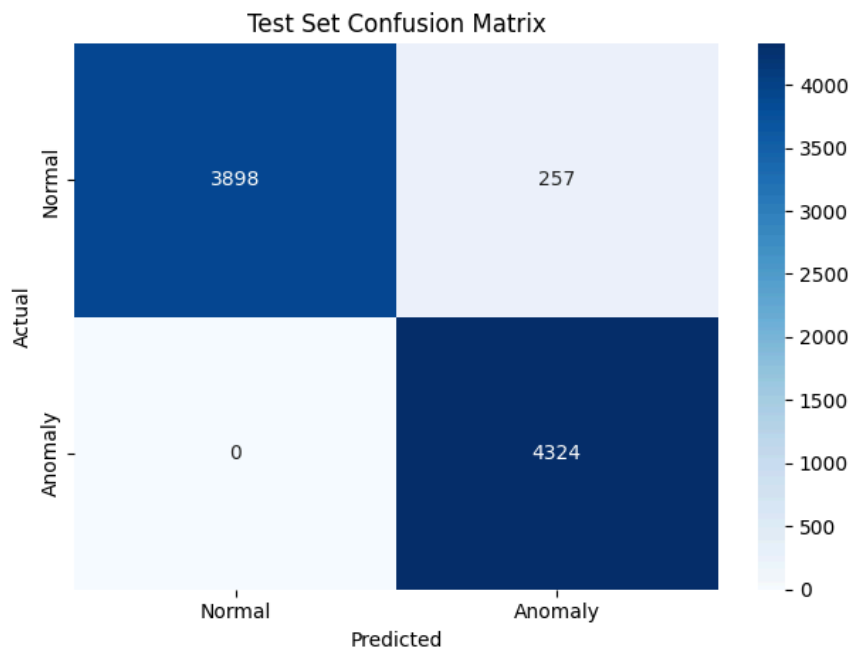
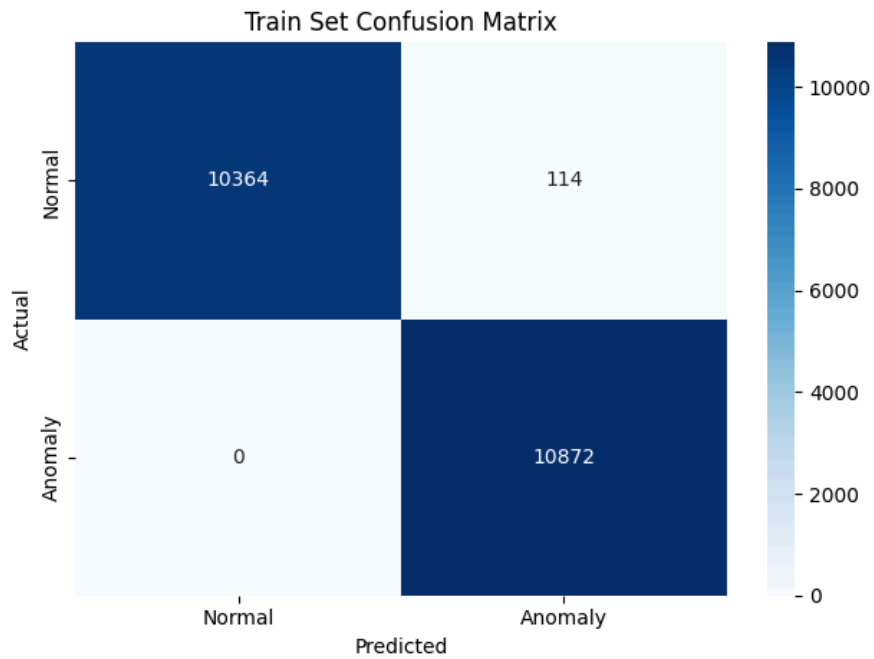
- UMAP visualization



- Reconstruction error distribution



- Selected anomaly threshold
Threshold = 0.8
I chose this value after experimenting with various threshold values (0.05, 0.6, 0.7, etc.). Based on my experiments, a threshold of 0.8 seemed to have the best precision and recall values.
- Confusion matrix



- Classification report or evaluation metrics

Train set evaluation				
	precision	recall	f1-score	support
Normal (0)	1.00	0.99	0.99	10478
Anomaly (1)	0.99	1.00	1.00	10872
accuracy			1.00	21350
macro avg	1.00	0.99	1.00	21350
weighted avg	1.00	1.00	1.00	21350
Test set evaluation				
	precision	recall	f1-score	support
Normal (0)	1.00	0.94	0.97	4155
Anomaly (1)	0.95	1.00	0.97	4324
accuracy			0.97	8479
macro avg	0.97	0.97	0.97	8479
weighted avg	0.97	0.97	0.97	8479

- Arduino serial monitor output showing reconstruction error and normal/anomaly detection results

```

Result: Normal activity
Reconstruction error: 0.092908
Result: Normal activity
Reconstruction error: 0.092905
Result: Normal activity
Reconstruction error: 0.092908
Result: Normal activity
Reconstruction error: 0.092913
Result: Normal activity
Reconstruction error: 0.092908
Result: Normal activity
Reconstruction error: 6.989710
Result: Anomaly detected
Reconstruction error: 7.399431
Result: Anomaly detected
Reconstruction error: 7.383115
Result: Anomaly detected
Reconstruction error: 7.629810
Result: Anomaly detected
Reconstruction error: 7.764740
Result: Anomaly detected

```

Part 3

- What threshold did you choose for anomaly detection, and why?

- I used a threshold of 0.8 for anomaly detection. I chose this value because it had relatively high precision and recall scores for the training and test sets. I experimented with various threshold values ranging from 0.05 to 0.9, and 0.8 had the best results.
- How does reconstruction error help distinguish normal and anomalous activity?
 - The autoencoder is trained on normal activity data, so it learns to efficiently compress and reconstruct normal motion patterns. When a normal window is passed through, the model reproduces it accurately, illustrated with a low mean squared error between input and output. When an anomalous window is passed through, the model tries to reconstruct it using the "normal" representation, causing the reconstruction to be poor and the MSE to be high. The threshold then acts as a decision boundary in which errors above it are flagged as anomalies.
- What information from the Python notebook must be preserved when deploying the model to Arduino?
 - The threshold value (0.8): the same cutoff must be hardcoded in the sketch
 - Feature columns (left ankle accelerometer axes 6, 7, 8): the Arduino must read the same sensor axes
 - Window size (100 timesteps) and step size: the sketch must buffer the same amount of data before running inference
 - Input dimensions ($300 = 100 \times 3$): the model's input tensor expects exactly this shape
 - The autoencoder_model.cc file: the quantized int8 TFLite model which contains all the trained weights and architecture in a format the Arduino can execute.
- Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?
 - It is important for the Arduino sketch to use the same preprocessing assumptions as the notebook because the TFLite model's weights were optimized during training to expect inputs in a specific format. If we don't follow the same axes, window length, and numerical scale, the input tensor will contain a completely different distribution than what the model was trained on. This would cause the model to flag everything as anomalous or not even though the model weights themselves are correct.
- What are the main limitations of this anomaly detection method when deployed on a microcontroller?
 - Fixed normal class definition: the model only knows what "normal" looked like at training time. New normal behaviors (a different walking style, a new user) will be incorrectly flagged.
 - No real IMU input: the sketch runs inference on static test data rather than live sensor streams, so real-world performance on a moving body is unvalidated.
 - Fixed sensor placement: the model was trained on a left ankle accelerometer. If we were to attach the Arduino somewhere else on the body, it might produce a different signal distribution.

- Threshold sensitivity: a single global threshold doesn't adapt to inter-user variability or sensor drift over time. A "normal" value for one person may not be normal for another person.